

Community Platform (Forum, Groups, Moderation)

1. Feature Name

Community Platform (Forum, Groups, Moderation)

2. Purpose

- **Problem Solved:** Learners and instructors need a place to discuss, ask questions, share code, and build study communities.
 - **Why It Exists:** The community layer provides posts with threaded comments and voting, public/private study groups, and an admin moderation pipeline (report → resolve/dismiss) to keep content safe.
 - **Role in Application:** It is the social subsystem. It operates independently of learning content but can link posts to learning paths; group posts add a scoped-community layer on top of the forum.
-

3. What the User Can Do

All users (Student / Instructor / Admin)

- Browse, search, filter, and sort community posts (newest / score / pinned / trending).
- Read a post (increments the view count) and its nested comment tree.
- Create / edit / delete their own posts (with title, content, tags, category, optional code snippet + language, optional learning-path link).
- Comment and reply on posts; edit/delete own comments.
- Upvote / downvote posts and comments, or withdraw a vote.
- Report a post or comment with a reason.
- Join public groups, leave groups, view group pages and group posts.
- Create group posts (membership required; banned users blocked).

Group owner / Admin

- Update/delete groups.
- Pin a group post as the single pinned announcement (only one pinned post per group).
- Ban / unban group members.

Admin only

- Create groups (`CreateGroupAsync` requires Admin).
 - Unpin posts outside groups.
 - View the pending moderation queue and resolve/dismiss reports.
-

4. Feature Workflow

[User posts a question]

- POST /api/v1/community/posts (validated by CreatePostValidator)
- CommunityService.CreatePostAsync → Post row created → POST_CREATED audit

[User views + votes]

- GET /community/posts/{id} → ViewCount++ persisted
- POST /community/posts/{id}/vote { isUpvote } → score math applied

[User comments / replies]

- POST /community/posts/{id}/comments { content, parentCommentId? }
- Parent verified within the same post; locked posts reject comments
- Comment row created (replies nested via ParentCommentId)

[User reports]

- POST /community/posts/{id}/report { reason }
- Report row created (Status = "Pending"); REPORT_CREATED audit

[Admin moderates]

- GET /community/reports (Admin only) → pending queue
- POST /community/reports/{id}/resolve | /dismiss

5. How It Works Internally

Post Listing, Filtering, and Sorting

GetPostsAsync(category, search, tag, sort, filter, page, pageSize, userId):

- Applies filters: category (skip "All"), search (title, content, tags, author name `Contains`), tag (in `Tags` string).
- `PostFilter` enum: `All`, `Mine` (author == user), `Pinned` (`IsPinned`), `WithCode` / `WithoutCode` (`CodeSnippet` null check).
- Non-trending ordering: `Score` sort → `OrderByDescending(Score).ThenByDescending(CreatedAt)`; otherwise `OrderByDescending(IsPinned).ThenByDescending(CreatedAt)`.
- Pagination: `Skip((page-1)*pageSize).Take(pageSize)`; `TotalPages = ceil(total/pageSize)`.

Trending Ranking

CalculateTrendingScore(int score, DateTime createdAt) (static, in `CommunityService`):

```
ageHours = max((UtcNow - createdAt).TotalHours, 0)
trending = score / (ageHours + 2) ^ 1.5
```

- Newer posts get a smaller age penalty; higher score wins. The denominator `(ageHours + 2)^1.5` never divides by zero and dampens old posts.
- Implementation detail: `GetTrendingPostsAsyncInternal` fetches the top candidates (`OrderByDescending(Score).Take(max(pageSize*10, 100))`), computes the trending score **in memory**, reorders, then applies page slicing.

Post Detail & View Counting

`GetPostByIdAsync` loads the post with author, votes, learning path, comments (with authors + votes + nested replies), increments `ViewCount`, saves, and returns the post with top-level comments (`ParentCommentId == null`) ordered newest first.

Voting Score Math

`VotePostAsync` (posts; comments use the identical logic in `VoteCommentAsync`):

- The user's existing vote on the target is checked.
- **Same direction as existing vote** → withdraw: remove the vote row; delta `-1` (was upvote) or `+1` (was downvote).
- **Opposite direction** → flip: update the vote; delta `-2` (was upvote, now downvote) or `+2` (was downvote, now upvote).
- **No existing vote** → insert; delta `+1` (upvote) or `-1` (downvote).
- `post.Score += delta`; the endpoint returns the current upvote count.
- `RemovePostVoteAsync` reverses the stored vote's contribution.

Group posts and comments require an active membership (`EnsureActiveMembershipAsync`) before voting.

Comment Threading

`Comment.ParentCommentId` is nullable and self-references `Replies`. Top-level comments (`ParentCommentId == null`) are returned with their `Replies` recursively mapped via `MapComment`. Creating a reply verifies the parent exists within the same post.

Locked Posts

`AddCommentAsync` rejects comments on posts where `IsLocked == true` ("This post is locked."). (Locking itself has no toggle endpoint found in the controller; the flag is set on the entity but is only enforced.)

Group Membership Model

`GroupMember.Role` string: "Owner" (creator), "Member", "Banned".

- Creating a group (Admin only) auto-adds the creator as `Owner`.
- Joining requires `group.IsPublic`; duplicate membership rejected.
- Leave is blocked for the owner ("The group owner cannot leave their own group.").
- Ban sets `Role = "Banned"`; unban restores "Member". The owner cannot be banned.
- `EnsureActiveMembershipAsync` blocks non-members and banned users from posting/commenting/voting in a group.

Group Moderation / Pinning

`EnsureGroupModeratorAsync(group, userId)` — passes for the group owner or any Admin.

`PinPostAsync`:

- Only posts **inside a group** can be pinned ("Only posts inside a group can be pinned.").
- Unpins all other pinned posts in the same group first (one pinned post per group), then pins the target.

`UnpinPostAsync`: group posts require a moderator; non-group posts require an Admin.

Reports & Moderation Queue

- `ReportPostAsync / ReportCommentAsync` create Report rows (`TargetType = "Post" | "Comment"`, `Status = "Pending"`, reason required).
- `GetModerationQueueAsync` (Admin only) lists Pending reports ordered by `CreatedAt` descending with pagination, resolving the reporter's name.
- `ResolveReportAsync` → `Status = "Resolved"`; `DismissReportAsync` → `Status = "Dismissed"`.

Post Summaries

`MapToSummary` builds `ContentPreview` = first 200 chars + "..."; `UserVote` = 1 (up), -1 (down), 0 (none);
`AuthorIsDeleted` = author status `Deleted`. `CommentCount` = `post.Comments.Count`.

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/community/CommunityPage.tsx</code>	Post list with filters/tabs, search, create-post modal
Page	<code>frontend/src/pages/community/CommunityPostPage.tsx</code>	Post detail with nested comments/replies and voting
Page	<code>frontend/src/pages/community/CommunityMyPostsPage.tsx</code>	Current user's posts
Page	<code>frontend/src/pages/community/CommunityGroupsPage.tsx</code>	Group list + create/join
Page	<code>frontend/src/pages/community/CommunityGroupDetailPage.tsx</code>	Group detail, group posts, members, ban/pin
Page	<code>frontend/src/pages/community/CommunityModerationPage.tsx</code>	Admin report/moderation queue
Component	<code>frontend/src/components/community/PostCard.tsx</code>	Post summary card
Component	<code>frontend/src/components/community/CommentItem.tsx</code>	Nested comment/thread component
Component	<code>frontend/src/components/community/CodeBlock.tsx</code>	Code snippet rendering (prism-react-renderer)
Component	<code>frontend/src/components/community/CreatePostModal.tsx</code>	Post creation/edit UI
Component	<code>frontend/src/components/community/ReportDialog.tsx</code>	Report modal
Service	<code>frontend/src/services/communityService.ts</code>	Post/comment/vote/group/report API calls

Layer	File Path	Responsibility
State	frontend/src/redux/slices/communitySlice.ts	Community Redux state
Hooks	frontend/src/hooks/useCommunity.ts, useGroups.ts	Community/group logic hooks

7. Backend Implementation

Layer	File Path	Responsibility
Controller	backend/Controllers/CommunityController.cs	Posts, comments, votes, groups, reports, pinning endpoints + validators/pagination
Service	backend/Services/Community/CommunityService.cs	Business rules, voting math, trending, group/moderation logic
Repository	backend/Repositories/CommunityRepository.cs	Group/post search and paged queries
Repository	backend/Repositories/GenericRepository.cs	Generic paged repository base
Repository	backend/Repositories/PagedResult.cs	Paged result container
Enum	backend/Repositories/PostSortOrder.cs	Newest / Score / Pinned / Trending
Enum	backend/Repositories/PostFilter.cs	All / Mine / Pinned / WithCode / WithoutCode
DTO	backend/DTOs/Community/*	Request/response DTOs
Validator	backend/Validators/Community/*	9 FluentValidation validators (post, comment, group, vote, report)
Interface	backend/Interfaces/Services/ICommunityService.cs	Service contract

8. Database Implementation

Entity/Table	Purpose	Important Relationship
Posts	Forum/group posts	FK AuthorId → Users; optional FK LearningPathId, GroupId; 1→N Comments, 1→N PostVotes; Score, ViewCount, IsPinned, IsLocked, tags, code
Comments	Nested comments/replies	FK PostId, FK AuthorId; self-FK ParentCommentId → Comments; 1→N CommentVotes

Entity/Table	Purpose	Important Relationship
PostVotes	Post up/down votes	Composite (PostId, UserId); IsUpvote
CommentVotes	Comment up/down votes	Composite (CommentId, UserId); IsUpvote
Groups	Study groups (public/private)	FK OwnerId → Users; 1→N GroupMembers, 1→N Posts
GroupMembers	Group membership + role	Composite (GroupId, UserId); Role = Owner/Member/Banned
Reports	Moderation reports	FK ReportedByUserId → Users; TargetType, TargetId, Status (Pending/Resolved/Dismissed)

- Migrations 20260731034404_AddGroupIdToPost.cs, 20260731050724_AddCommunityModeration.cs, 20260731062500_AddPostCodeSnippetAndTags.cs added the community-specific schema.

9. Security & Authorization

- Post/comment/group/vote/report create/update operations require authentication ([Authorize]); post listing/detail and group listing are [AllowAnonymous] (partial viewing without auth).
- Editing a post requires ownership (GetOwnedPostAsync); deleting can be done by the author **or an Admin** (GetOwnedPostOrAdminAsync).
- Editing a comment requires ownership; deleting by the author or any Admin.
- Group create/moderation/pinning/report resolution require Admin or group-owner privileges (EnsureAdminAsync, EnsureGroupModeratorAsync).
- Group posts/comments/votes require active membership; banned users are rejected.
- All validators run before service calls; pagination bounds (page ≥ 1, pageSize 1–100) are enforced in the controller.

10. Important Business Rules

1. **Voting math:** identical re-vote = withdraw; opposite vote = flip (delta ±2); first vote = ±1.
2. **Trending score** = $\text{score} / (\text{ageHours} + 2)^{1.5}$, computed over a $\max(\text{pageSize} \times 10, 100)$ candidate window.
3. **Comment replies** must target a comment within the same post; locked posts reject new comments.
4. **Group joining** only for public groups; owner cannot leave; owner cannot be banned.
5. **Pinning:** one pinned post per group (pinning others unpins the previous); only group posts can be pinned.
6. **Reports:** a reason is required; only Admin can resolve/dismiss.
7. **Content preview** truncated to 200 characters.
8. **Deleted authors** are flagged (AuthorIsDeleted) rather than blocking content.

11. Example of Internal Execution

Step 1: User creates a post and upvotes it

- **User Action:** User writes a post and then upvotes it.
- **Frontend:** `communityService.createPost` → `POST /community/posts`; then `POST /community/posts/42/vote { isUpvote: true }`.
- **Service:** `CreatePostAsync` stores the post (score 0). `VotePostAsync` finds no existing vote → inserts `PostVote (42, user, up)` and `Score += 1`.
- **Response:** upvote count `1`.

Step 2: User upvotes the same post again

- Existing vote direction matches → the vote row is removed, `Score -= 1` (back to 0), a `VOTE_REMOVED` audit is written. This makes voting a toggle.

Step 3: Trending listing

- `GET /community/posts?sort=Trending` — the service pulls top-scored candidates, runs `CalculateTrendingScore` per post, reorders, then pages.

Step 4: Group + moderation

- An Admin creates a group and pins an announcement (`PinPostAsync` unpins any other pinned post in the group).
- A user reports an abusive comment → `Report` row (Pending).
- Admin opens the moderation queue, the report appears, and `ResolveReportAsync` sets `Status = "Resolved"`.

12. Files Involved

Frontend

- `frontend/src/pages/community/CommunityPage.tsx`
- `frontend/src/pages/community/CommunityPostPage.tsx`
- `frontend/src/pages/community/CommunityMyPostsPage.tsx`
- `frontend/src/pages/community/CommunityGroupsPage.tsx`
- `frontend/src/pages/community/CommunityGroupDetailPage.tsx`
- `frontend/src/pages/community/CommunityModerationPage.tsx`
- `frontend/src/components/community/*`
- `frontend/src/services/communityService.ts`
- `frontend/src/redux/slices/communitySlice.ts`
- `frontend/src/hooks/useCommunity.ts, useGroups.ts`

Backend

- `backend/Controllers/CommunityController.cs`
- `backend/Services/Community/CommunityService.cs`

- `backend/Repositories/CommunityRepository.cs`
- `backend/Repositories/GenericRepository.cs`
- `backend/Repositories/PostSortOrder.cs`
- `backend/Repositories/PostFilter.cs`
- `backend/Repositories/PagedResult.cs`
- `backend/DTOs/Community/*`
- `backend/Validators/Community/*`
- `backend/Interfaces/Services/ICommunityService.cs`

Database

- `backend/Entities/Post.cs`
- `backend/Entities/Comment.cs`
- `backend/Entities/PostVote.cs`
- `backend/Entities/CommentVote.cs`
- `backend/Entities/Group.cs`
- `backend/Entities/GroupMember.cs`
- `backend/Entities/Report.cs`
- `backend/Migrations/20260731034404_AddGroupIdToPost.cs`
- `backend/Migrations/20260731050724_AddCommunityModeration.cs`
- `backend/Migrations/20260731062500_AddPostCodeSnippetAndTags.cs`

13. Functionality

- Post creation/editing/deletion with tags, categories, code snippets, learning-path links
- Post listing with filters (mine, pinned, with/without code), search, and sorting (newest, score, pinned, trending)
- Trending rank calculation ($\text{score} \div \text{age-decay}$)
- Post view counting
- Nested comment threads with replies
- Post and comment up/down voting with toggle/flip logic
- Code snippet rendering with syntax highlighting
- Public/private study groups (join/leave/owner rules)
- Group posts (membership-gated, banned-user blocked)
- Group member ban/unban
- Single pinned group announcement
- Reporting posts/comments
- Admin moderation queue (resolve/dismiss)
- Audit logging across post/comment/vote/group/report actions